



BECOME JAVA EXPERT

In Just 20 Days





Introduction and Setting Goals

GOALS

Understand the basics of Java and set specific learning goals for the next 20 days.

TOPICS

- Introduction to Java programming language
 - Setting up the Java development environment (JDK, IDE)
 - Understanding the Java Virtual Machine (JVM)
-

RESOURCES

<https://docs.oracle.com/javase/tutorial/>
Eclipse or IntelliJ IDEA (Java IDE)



Practice Questions

1. What is Java programming language and what are its key features?
2. Explain the process of setting up the Java development environment, including the installation of JDK and an IDE.
3. What is the Java Virtual Machine (JVM) and what is its role in executing Java programs?
4. Describe the steps involved in compiling and running a Java program using the command line.
5. How does Java handle memory management and what is the significance of the garbage collector?
6. Compare and contrast Eclipse and IntelliJ IDEA as Java IDEs, highlighting their key features and differences.
7. What are the basic data types available in Java, and how are they used in variable declarations and assignments?



Java Syntax and Variables

GOALS

Gain a solid understanding of Java syntax and learn about variables.

TOPICS

- Java program structure (classes, methods, statements)
 - Data types and variables in Java
 - Variable declaration, initialization, and scope
-

RESOURCES

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html>



Practice Questions

1. What is the basic structure of a Java program? Describe the roles of classes, methods, and statements.
2. What are the primitive data types in Java? Give examples of each.
3. How do you declare and initialize a variable in Java? Provide an example for each of the following data types: int, double, boolean, and String.
4. What is the scope of a variable in Java? Explain the concepts of local variables, instance variables, and class variables.
5. What is the difference between the "==" operator and the ".equals()" method when comparing two objects in Java?
6. Explain the concept of type casting in Java. How can you convert a variable from one data type to another?
7. What is the difference between static and non-static methods in Java? How do you call each of these methods?



Operators and Expressions

GOALS

Learn about operators and expressions in Java.

TOPICS

- Arithmetic, assignment, and comparison operators
 - Conditional and logical operators
 - Precedence and associativity of operators
-

RESOURCES

- https://www.w3schools.com/java/java_operators.asp
- <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/operators.html>



Practice Questions

1. 1. What will be the result of the following expression?

```
int x = 10;  
int y = 3;  
int result = x % y;  
System.out.println(result);
```
2. Which operator is used for exponentiation in Java?
3. Consider the following code snippet:

```
int a = 5;  
int b = 7;  
int c = 3;  
boolean result = (a > b) && (b < c);  
System.out.println(result);
```

What will be the output of this code?
4. What is the value of the expression "Hello" + "World" in Java?
5. Which operator is used to assign a value to a variable in Java?
6. What will be the value of the expression $5 * 3 + 2 - 4 / 2$ in Java?
7. Consider the following code snippet:

```
int x = 5;  
int y = 3;  
int z = 7;  
boolean result = (x < y) || (y > z);  
System.out.println(result);
```



Control Flow Statements

GOALS

Understand control flow statements and their usage in Java.

TOPICS

- Conditional statements (if-else, switch)
 - Looping statements (for, while, do-while)
 - Branching statements (break, continue, return)
-

RESOURCES

- https://www.w3schools.com/java/java_while_loop.asp
- <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/flow.html>



Practice Questions

1. What is the purpose of a conditional statement in Java, and what are the two main types of conditional statements?
2. Write a Java code snippet that uses an if-else statement to check if a given number is even or odd. If the number is even, print "Even"; otherwise, print "Odd."
3. What is the purpose of a switch statement in Java, and how is it different from an if-else statement?
4. Write a Java code snippet that uses a switch statement to display the name of a day of the week based on a given integer input. For example, if the input is 1, the code should print "Monday."
5. What is the purpose of a loop statement in Java, and what are the three main types of loop statements?
6. Write a Java code snippet that uses a for loop to calculate the sum of all numbers from 1 to 10 and prints the result.
7. What is the difference between the break, continue, and return statements in Java? Explain their usage with an example for each.



Arrays and Strings

GOALS

Learn about arrays and strings in Java.

TOPICS

- Declaring and initializing arrays
 - Array operations (accessing elements, length, sorting)
 - String manipulation and methods
-

RESOURCES

- https://www.w3schools.com/java/java_arrays.asp
- <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/arrays.html>
- <https://docs.oracle.com/javase/tutorial/java/data/strings.html>



Practice Questions

1. 1. What is the correct way to declare an array of integers named "numbers" with a length of 5?
 - a. `int[] numbers = new int[5];`
 - b. `int numbers[5];`
 - c. `int[5] numbers;`
 - d. `int numbers = new int[5];`
2. How do you access the element at index 2 in an array named "myArray"?
 - a. `myArray.get(2);`
 - b. `myArray[2];`
 - c. `myArray.getElement(2);`
 - d. `myArray.getElementAt(2);`
3. What is the length of the following array: `String[] names = {"John", "Mary", "Peter", "Lisa"};`
 - a. 3
 - b. 4
 - c. 5
 - d. It will throw an error.



Practice Questions

4. Which method is used to sort an array in ascending order in Java?
- a. `sortArray()`
 - b. `sort()`
 - c. `sortAscending()`
 - d. `sortArrayAscending()`
5. Which operator is used to concatenate two strings in Java?
- a. `+`
 - b. `*`
 - c. `&`
 - d. `/`
6. What is the output of the following code snippet?
- ```
String str = "Hello World!";
System.out.println(str.length());
```
- a. Hello World!
  - b. Hello
  - c. 12
  - d. 11





# Practice Questions

7. Which method is used to convert an integer to a string in Java?
- a. toString()
  - b. parseInt()
  - c. toInteger()
  - d. convertToString()





# Object-Oriented Programming (OOP) Concepts

## GOALS

Understand the core principles of object-oriented programming in Java.

---

## TOPICS

- Classes and objects
  - Encapsulation, inheritance, and polymorphism
  - Constructors and method overloading
- 

## RESOURCES

- <https://www.javatpoint.com/java-oops-concepts>
- [https://www.w3schools.com/java/java\\_oop.asp](https://www.w3schools.com/java/java_oop.asp)
- <https://docs.oracle.com/javase/tutorial/java/concepts/index.html>





# Practice Questions

1. What is the purpose of encapsulation in object-oriented programming? Provide an example of how encapsulation can be achieved in Java.
2. Explain the concept of inheritance in Java. How does inheritance promote code reuse and maintainability?
3. What is polymorphism in Java? Give an example of how polymorphism can be implemented using method overriding.
4. What is the difference between method overloading and method overriding in Java? Provide an example to illustrate each concept.
5. How are objects created in Java? What is the difference between a class and an object?
6. Explain the role of constructors in Java. What is the difference between a default constructor and a parameterized constructor?
7. How can you achieve multiple inheritance in Java? Is it supported by the language? If not, how can you simulate multiple inheritance using interfaces?



# Class Inheritance and Interfaces

## GOALS

Dive deeper into class inheritance and interfaces in Java.

---

## TOPICS

- Single and multiple inheritance in Java
  - Abstract classes and methods
  - Interfaces and their implementation
- 

## RESOURCES

- [https://www.w3schools.com/java/java\\_classes.asp](https://www.w3schools.com/java/java_classes.asp)
- <https://docs.oracle.com/javase/tutorial/java/landl/subclasses.html>
- <https://docs.oracle.com/javase/tutorial/java/landl/createinterface.html>



# Practice Questions

1. What is the difference between single and multiple inheritance in Java? Provide an example of each.
2. Explain what an abstract class is in Java and how it is different from a regular class. Provide an example of an abstract class and its usage.
3. How can you achieve multiple inheritance-like behavior in Java when it only supports single inheritance? Provide an example using interfaces.
4. What is the purpose of an interface in Java? How does it differ from an abstract class? Provide an example of an interface and its implementation.
5. Can abstract classes have constructors in Java? Explain why or why not, and provide an example to support your answer.
6. Can an interface extend another interface in Java? If so, what is the purpose of extending an interface? Provide an example of an interface that extends another interface.
7. When should you use inheritance and when should you use interfaces in Java? Explain the advantages and disadvantages of each approach, and give an example scenario for each.



# Exception Handling

## GOALS

Learn about exception handling and how to deal with errors in Java.

---

## TOPICS

- Understanding exceptions and error handling
  - The try-catch-finally block
  - Exception propagation and checked vs. unchecked exceptions
- 

## RESOURCES

- <https://www.geeksforgeeks.org/exceptions-in-java/>
- <https://docs.oracle.com/javase/tutorial/essential/exceptions/index.html>





# Practice Questions

1. What is an exception in Java, and why is it important to handle exceptions properly in a program?
2. Explain the purpose of the try-catch-finally block in Java. Provide an example of how it can be used to handle exceptions.
3. What is the difference between checked and unchecked exceptions in Java? Give examples of each.
4. Can you have multiple catch blocks for a single try block in Java? If yes, how are they executed? Provide an example.
5. What is exception propagation in Java? Explain how exceptions are propagated up the call stack.
6. How can you create custom exceptions in Java? Provide an example of a custom exception class.
7. Explain the concept of the "finally" block in Java exception handling. What is its purpose, and when is it executed?



# Input and Output (I/O) Operations

## GOALS

Gain knowledge about I/O operations in Java.

---

## TOPICS

- Reading and writing to the console
  - File I/O operations (reading, writing, and manipulating files)
  - Serialization and deserialization of objects
- 

## RESOURCES

- <https://www.geeksforgeeks.org/java-io-input-output-in-java-with-examples/>
- <https://docs.oracle.com/javase/tutorial/essential/io/index.html>



# Practice Questions

1. What is the purpose of I/O operations in Java?
2. How can you read input from the console in Java? Provide an example code snippet.
3. How can you write output to the console in Java? Provide an example code snippet.
4. What are the steps involved in performing file I/O operations in Java?
5. How can you read the contents of a file in Java? Provide an example code snippet.
6. How can you write data to a file in Java? Provide an example code snippet.
7. What is serialization and deserialization in Java? How can you implement it for objects?





# Collections Framework

## GOALS

Understand the Java Collections Framework and its various data structures.

---

## TOPICS

- Introduction to the Collections Framework
  - Lists, Sets, and Maps
  - Iterators and the enhanced for loop
- 

## RESOURCES

- <https://docs.oracle.com/javase/tutorial/collections/index.html>



# Practice Questions

1. What is the Java Collections Framework, and what are its main goals?
2. What is the difference between a List and a Set in the Java Collections Framework? Provide an example of when each data structure would be appropriate to use.
3. What is the purpose of the Map interface in the Java Collections Framework? Give an example of a real-world scenario where a Map would be useful.
4. Explain the concept of an iterator in the context of the Java Collections Framework. How is it different from using the enhanced for loop?
5. What is the time complexity for retrieving an element from an ArrayList in the Java Collections Framework? Is it the same for a LinkedList? Justify your answer.
6. In which situations would you prefer to use a HashSet over a TreeSet, and vice versa?
7. What are some benefits of using the Java Collections Framework over manually implementing data structures in your code?



## Generics

### GOALS

Learn about generics and how to use them in Java.

---

### TOPICS

- Introduction to generics
  - Generic classes, methods, and interfaces
  - Type erasure and bounded type parameters
- 

### RESOURCES

- <https://docs.oracle.com/javase/tutorial/java/generics/index.html>



# Practice Questions

1. What are generics in Java, and what problem do they solve?
2. How do you declare a generic class in Java? Provide an example.
3. Can you explain the concept of type erasure in Java generics? How does it work?
4. How can you use bounded type parameters in generics? Give an example.
5. Is it possible to use primitive types as type arguments in Java generics? Why or why not?
6. How do you define a generic method in Java? Provide an example.
7. Can you implement a generic interface in Java? If so, how? Give an example.





# Multithreading

## GOALS

Understand multithreading concepts and how to create concurrent programs in Java.

---

## TOPICS

- Introduction to multithreading
  - Creating and managing threads
  - Synchronization and thread safety
- 

## RESOURCES

- <https://www.geeksforgeeks.org/multithreading-in-java/?ref=lbp>
- <https://docs.oracle.com/javase/tutorial/essential/concurrency/index.html>



# Practice Questions

1. What is multithreading? Explain the concept with an example.
2. How do you create a thread in Java? Provide code snippets to illustrate the process.
3. What is the difference between extending the Thread class and implementing the Runnable interface when creating a thread in Java?
4. What is thread synchronization? Why is it important in multithreaded programming?
5. What are the ways to achieve thread synchronization in Java? Compare and contrast the synchronized keyword and the ReentrantLock class.
6. Explain the concept of thread safety. How can you ensure thread safety in Java?
7. What are the advantages and disadvantages of multithreading in Java? Provide examples of scenarios where multithreading is beneficial and when it might not be suitable.



# Lambda Expressions and Functional Programming

## GOALS

Learn about lambda expressions and functional programming in Java.

---

## TOPICS

- Introduction to lambda expressions
  - Functional interfaces and the `java.util.function` package
  - Method references and stream API
- 

## RESOURCES

- <https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html>





# Practice Questions

1. What is a lambda expression in Java and how is it different from a regular method?
2. Explain the concept of functional interfaces in Java. How are they related to lambda expressions?
3. How can lambda expressions be used with the `java.util.function` package? Provide an example of a functional interface from this package.
4. What are method references in Java? How do they simplify the usage of lambda expressions?
5. Describe the purpose of the Stream API in Java. How can lambda expressions be utilized with streams?
6. Can lambda expressions access variables from their surrounding scope? Explain with an example.
7. Discuss the advantages of using lambda expressions and functional programming in Java compared to traditional imperative programming paradigms.



## JDBC and Database Connectivity

### GOALS

Understand how to connect Java applications to databases using JDBC.

### TOPICS

- Introduction to JDBC and database connectivity
- Establishing database connections
- Executing SQL queries and processing results

### RESOURCES

- <https://www.javatpoint.com/steps-to-connect-to-the-database-in-java#:~:text=There%20are%205%20steps%20to%20connect%20any%20java,Create%20connection%20Create%20statement%20Execute%20queries%20Close%20connection>
- <https://docs.oracle.com/javase/tutorial/jdbc/index.html>



# Practice Questions

1. What is JDBC and what is its role in Java applications?
2. What are the steps involved in establishing a database connection using JDBC?
3. How do you handle exceptions when establishing a database connection in JDBC?
4. How do you execute SQL queries using JDBC? Provide an example.
5. What are the different types of JDBC statements? Explain each one briefly.
6. How can you retrieve and process the result set obtained from a JDBC query?
7. What is connection pooling in JDBC? How does it improve performance in database connectivity?



# GUI Development with Swing

## GOALS

Gain knowledge of GUI development using the Swing framework in Java.

---

## TOPICS

- Introduction to Swing and GUI components
  - Building UI components and layouts
  - Event handling and listeners
- 

## RESOURCES

- <https://www.javatpoint.com/java-swing>
- <https://docs.oracle.com/javase/tutorial/uiswing/index.html>





# Practice Questions

1. What is Swing in Java and how does it differ from AWT?
2. Explain the concept of GUI components in Swing.  
Provide examples of commonly used Swing components.
3. How do you create a JFrame in Swing? Describe the steps involved in creating a basic JFrame with a title and size.
4. What are layout managers in Swing? Name and briefly explain three different layout managers available in Swing.
5. What is event handling in Swing? How do you register an event listener for a specific component in Swing?
6. Explain the concept of event propagation in Swing. What is the order in which events are propagated through Swing components?
7. How do you handle button click events in Swing?  
Provide an example of adding an ActionListener to a JButton and performing an action when the button is clicked.



# JavaFX and Modern UI Development

## GOALS

Explore JavaFX for modern UI development in Java.

---

## TOPICS

- Introduction to JavaFX and its advantages over Swing
  - Creating UI using FXML and CSS
  - Handling events and animations in JavaFX
- 

## RESOURCES

- <https://openjfx.io/documentation/>



# Practice Questions

1. What are some advantages of JavaFX over Swing for modern UI development in Java?
2. How can you create a user interface using FXML in JavaFX? Explain the steps involved.
3. What is the role of CSS in JavaFX? How can you apply CSS styles to JavaFX components?
4. How can you handle events in JavaFX? Provide an example of adding an event handler to a button.
5. What are animations in JavaFX? How can you create animations for UI elements?
6. Can you explain the concept of binding in JavaFX? How does it help in synchronizing UI elements?
7. What are some commonly used layout containers in JavaFX? Briefly describe their purpose and when to use them.



# Unit Testing with JUnit

## GOALS

Learn about unit testing and how to use JUnit for testing Java code.

---

## TOPICS

- Introduction to unit testing and its importance
  - Writing JUnit test cases
  - Test suites and annotations in JUnit
- 

## RESOURCES

- <https://junit.org/junit5/docs/current/user-guide/>





# Practice Questions

1. What is unit testing and why is it important in software development?
2. What is the purpose of JUnit in Java unit testing?
3. How do you write a basic JUnit test case in Java?
4. What are assertions in JUnit? How are they used to verify expected behavior in test cases?
5. What is a test suite in JUnit? How can you create and run multiple test cases using a test suite?
6. What are JUnit annotations? Provide examples of commonly used annotations in JUnit.
7. How can you perform exception handling in JUnit test cases? Explain with an example.



# Advanced Java Concepts

## GOALS

Explore advanced concepts and features of Java.

---

## TOPICS

- Reflection and dynamic class loading
  - Annotations and their usage
  - Java Native Interface (JNI)
- 

## RESOURCES

- <https://docs.oracle.com/javase/tutorial/reflect/index.html>



# Practice Questions

1. What is reflection in Java, and how does it enable dynamic class loading?
2. How do you use reflection to access and modify fields and methods of a class at runtime?
3. Explain the concept of annotations in Java and their usage in programming.
4. What are some built-in annotations in Java, and how are they commonly used?
5. How can you create your own custom annotations in Java, and what are some practical use cases for them?
6. What is the Java Native Interface (JNI), and how does it facilitate communication between Java and native code?
7. Explain the steps involved in using JNI to call native code from a Java application.



# Java Best Practices and Code Quality

## GOALS

Understand best practices and techniques for writing efficient and maintainable Java code.

---

## TOPICS

- Writing clean and readable code
  - Using design patterns and SOLID principles
  - Code documentation and commenting
- 

## RESOURCES

- Effective Java by Joshua Bloch (book)





# Practice Questions

1. What is the importance of writing clean and readable code in Java? How does it contribute to code quality and maintenance?
2. How can you ensure code quality in Java by following the SOLID principles? Provide an example of how you can apply one of the SOLID principles in Java code.
3. Why is it important to use design patterns in Java? Give an example of a commonly used design pattern and explain how it improves code quality and maintainability.
4. Explain the benefits of documenting your Java code. What information should be included in code documentation, and how can it help other developers understand and use your code effectively?
5. What are some best practices for writing meaningful comments in Java code? How can comments enhance code readability and assist in maintaining the codebase?



# Project Development and Real-World Applications

## GOALS

Apply the knowledge gained throughout the 20 days to develop a small Java project.

---

## TOPICS

- Project planning and requirements gathering
  - Designing and implementing a Java application
  - Testing, debugging, and deployment
- 

## RESOURCES

- <https://www.geeksforgeeks.org/java-projects/>



# Practice Questions

1. How do you plan and execute a Java project from start to finish?
2. What are the key considerations for designing and implementing a Java application?
3. How can you effectively test, debug, and deploy a Java project?





## WHY BOSSCODER?

 **750+** Alumni placed at Top Product-based companies.

 More than **136% hike** for every **2 out of 3** working professional.

 Average package of **24LPA.**

The syllabus is most up-to-date and the list of problems provided covers all important topics.

Lavanya  
 Meta



Course is very well structured and streamlined to crack any MAANG company

Rahul  




[\*\*EXPLORE MORE\*\*](#)